

Case Study: Library Management System

Hibernate / JPA (No Spring) — Maven Project

Objective

Build a simple Library Management System backend using plain Hibernate/JPA (Jakarta EE, Hibernate 6.x) in a Maven project. No Spring, no Spring Data — you will write your own `SessionFactory` setup and DAO classes by hand.

This exercise tests your understanding of:

- Entity mapping and relationships (`@ManyToOne`)
 - Enum mapping
 - CRUD operations (Create, Read, Update, Delete)
 - HQL/JPQL queries
 - Criteria API for dynamic filtering
 - One native SQL query (to understand when/why it's needed)
-

Entity Design

You must create **3 entity classes**: `Author`, `Book`, `Member`.

1. Author

Field	Type
id	Long (auto-generated)
name	String
country	String

2. Book

Field	Type
id	Long (auto-generated)
title	String
genre	enum Genre → FICTION, NON_FICTION, SCIENCE, HISTORY, BIOGRAPHY
status	enum BookStatus → AVAILABLE, BORROWED, LOST
author	@ManyToOne → Author (a book has exactly one author)
borrowedBy	@ManyToOne → Member (nullable — null when not currently borrowed)
publishedYear	int

3. Member

Field	Type
id	Long (auto-generated)
name	String
email	String (unique)
membershipType	enum MembershipType → STANDARD, PREMIUM
joinedAt	Instant (use @CreationTimestamp)

Relationship notes:

- Book → Author : Many books, one author
- Book → Member : Many books can (at different times) be borrowed by one member; nullable, since a book may not currently be borrowed.

Setup Requirements

- Plain Maven project, no Spring dependencies anywhere.
- Hibernate 6.x with Jakarta EE annotations.
- MySQL as the database.
- Programmatic SessionFactory configuration — **no hibernate.cfg.xml**.
- hbm2ddl.auto=update for schema generation during development.
- Follow the DAO pattern (one DAO class per entity — AuthorDAO, BookDAO, MemberDAO).

Tasks

Task 1 — Basic CRUD (all 3 entities)

For **each** of the 3 entities, implement:

- save(entity) — insert
- findById(id) — read single
- findAll() — read all
- update(entity) — update
- delete(id) — delete

Task 2 — Find by relationship (HQL/JPQL)

Write a method `findBooksByAuthor(Long authorId)` that returns all books written by a given author, using HQL with a join.

Write a method `findBooksBorrowedByMember(Long memberId)` that returns all books currently borrowed by a given member.

Task 3 — Dynamic Filtering (Criteria API)

Write a method:

```
List<Book> filterBooks(Genre genre, BookStatus status)
```

- Both parameters are **optional** (nullable) — filtering should work with either, both, or neither provided.
- Use the Criteria API (`CriteriaBuilder`, `CriteriaQuery`, `Root`, `Predicate` list) — not HQL — for this task.
- If both are null, return all books.

Constraints & Rules

- **No Spring, no Spring Data JPA, no Spring Boot** — plain Hibernate only.
- Prefer **HQL/JPQL** for all query tasks.
- Task 4 **must** use the Criteria API, not HQL — the point is to practice building dynamic, optional filter logic programmatically.
- Use `try-with-resources` for every `Session`.
- Handle transactions with rollback on failure (as shown in class).

Part 2: Many-to-Many Relationship — Library Events

Libraries run events (author talks, book clubs, workshops), and members can register for multiple events, while each event can have multiple registered members. This is a genuine many-to-many — and since we also need to track **when** a member registered and whether they actually **attended**, a plain `@ManyToMany` collection isn't enough. You need an explicit join entity.

New Entity 4 — `LibraryEvent`

Field	Type
<code>id</code>	Long (auto-generated)
<code>title</code>	String
<code>eventDate</code>	Instant

Field	Type
eventType	enum EventType → BOOK_CLUB, AUTHOR_TALK, WORKSHOP

New Entity 5 — EventRegistration (the join entity)

Field	Type
id	Long (auto-generated)
member	@ManyToOne → Member
event	@ManyToOne → LibraryEvent
registeredAt	Instant (@CreationTimestamp)
attendanceStatus	enum AttendanceStatus → REGISTERED, ATTENDED, NO_SHOW

Dev Tip:

Why a join entity instead of @ManyToMany directly: a plain @ManyToMany only manages the two foreign keys — it has nowhere to put `registeredAt` or `attendanceStatus`. Modeling the relationship as its own entity (`EventRegistration`) with two @ManyToOnes is the standard pattern once a many-to-many relationship needs to carry its own data.

Task 6:

Write a method `findRegistrationsForEvent(Long eventId)` that returns all `EventRegistration` rows for a given event

Task 7:

Write a method `findNoShowsForEvent(Long eventId)` that returns all `Members` who registered for a given event but did not attend — filtering directly on `EventRegistration.attendanceStatus`.

Task 8:

Create an `EventRegistrationDTO` and an `EventRegistrationMapper` class to convert `EventRegistration` entities to DTOs, and update the service-facing method to return `List<EventRegistrationDTO>` instead of `List<EventRegistration>`.